Invisible WiFi LED Strip

Make WiFi signal strength visible with an ESP32 and a WS2812B LED strip: wiring, setup and full code.

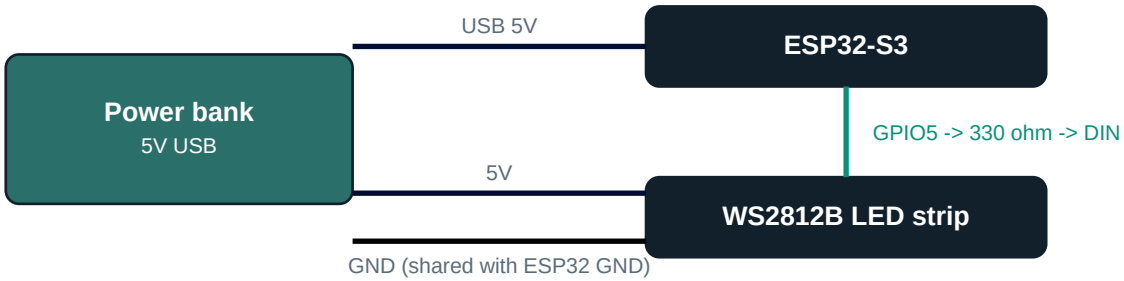
1. What it does

The ESP32 connects to a WiFi network and reads the signal strength (RSSI, in dBm) about every 20 ms. The reading is smoothed and shown as a bar on the LED strip. A strong signal fills the bar and turns it green. A weak signal shrinks the bar and turns it red.

2. Parts

Part	Notes
ESP32 or ESP32-S3 board	Any board with WiFi works. The sketch uses GPIO5 for the strip.
WS2812B LED strip	20 LEDs is a good size. Change NUM_LEDS in the code to match yours.
USB power bank	Powers both the ESP32 and the strip. Output must be 5V.
330 ohm resistor	In series with the data line (GPIO5 to DIN).
1000 uF capacitor (optional)	Across strip 5V and GND, protects the LEDs at power-up.
Jumper wires	Red = 5V, white/black = GND, green = data.

3. Wiring



LED strip	Connects to
5V	Power bank 5V (not the ESP32 3V3 pin)
GND	Power bank GND and ESP32 GND. The grounds must be common.
DIN (data in)	ESP32 GPIO5, through the 330 ohm resistor

Power the ESP32 from the same power bank through its USB port or 5V pin.

4. Setup

- Install the **FastLED** library in the Arduino IDE (Library Manager).
- Select the board **ESP32S3 Dev Module** (or your ESP32 board). Turn on **USB CDC On Boot** to see Serial output.
- Edit **WIFI_SSID** and **WIFI_PASS** at the top of the sketch. A phone hotspot works well.
- Set **NUM_LEDS** to the number of LEDs on your strip, then upload.

- Open the Serial Monitor at 115200 baud to see live RSSI values.

5. LED behavior

State	Strip shows
Connecting to WiFi	A blue light chases along the strip
WiFi lost	Slow red pulse; the ESP32 retries every 5 seconds
Connected	Bar length and color follow the signal (red = weak, green = strong)

6. Tips for filming and tuning

- In real life, a hand or a metal pan near the ESP32 usually changes the signal by only a few dB. Set **RSSI_WEAK = -75** and **RSSI_STRONG = -40** to make the effect clearly visible.
- Hold the metal close to the antenna end of the ESP32 for the biggest drop.
- If the strip flickers more than you like, lower **SMOOTHING** (for example 0.08). If it reacts too slowly, raise it.
- The ESP32 3.3V data signal normally drives WS2812B strips fine. If the LEDs flicker or show wrong colors, add a 74AHCT125 level shifter on the data line.
- Keep BRIGHTNESS low (around 80) when running from a power bank. The sketch also limits power draw to 500 mA.

7. Full code: wifi_led_visualizer.ino

Copy this into a new Arduino sketch. Not yet compiled on hardware: compile and test it on your own board first.

```
/*
  Invisible WiFi -> visible LED strip
  ESP32 / ESP32-S3 + WS2812B strip

  Wiring (matches the video):
    Strip 5V   -> power bank 5V (NOT the ESP32 3V3 pin)
    Strip GND  -> power bank GND AND ESP32 GND (common ground is required)
    Strip DIN  -> ESP32 GPIO5 through a 330 ohm resistor
    Power the ESP32 from the same power bank (5V pin / USB)
    Optional but recommended: 1000 uF capacitor across strip 5V and GND

  Library: FastLED (Arduino IDE -> Library Manager -> "FastLED")
  Board:   ESP32S3 Dev Module (or your ESP32 board), USB CDC On Boot = Enabled
*/

#include <WiFi.h>
#include <FastLED.h>

// ---------- EDIT THESE ----------
const char* WIFI_SSID = "YOUR_WIFI_NAME";      // phone hotspot or router
const char* WIFI_PASS = "YOUR_WIFI_PASSWORD";

#define LED_PIN     5       // data pin (IO5)
#define NUM_LEDS    20      // number of LEDs on your strip
#define BRIGHTNESS  80      // 0-255, keep low on a power bank

// RSSI range mapped to the bar. Typical: -90 dBm (weak) .. -35 dBm (strong).
// For a more dramatic demo in a small room, try -75 .. -40.
const int RSSI_WEAK   = -90;
const int RSSI_STRONG = -35;

// Smoothing: smaller = smoother but slower, bigger = faster but jumpier.
const float SMOOTHING = 0.15f;
// --------------------------------

CRGB leds[NUM_LEDS];
float level = 0.0f;                 // smoothed 0.0 .. 1.0
unsigned long lastReconnect = 0;

void showConnecting() {
  // blue chase while connecting
  static uint8_t pos = 0;
  fill_solid(leds, NUM_LEDS, CRGB::Black);
  leds[pos % NUM_LEDS] = CRGB::Blue;
  FastLED.show();
  pos++;
}

void showDisconnected() {
  // slow red pulse = no WiFi
  uint8_t b = beatsin8(40, 10, 120);
  fill_solid(leds, NUM_LEDS, CRGB(b, 0, 0));
  FastLED.show();
}

void setup() {
  Serial.begin(115200);

  FastLED.addLeds<WS2812B, LED_PIN, GRB>(leds, NUM_LEDS);
  FastLED.setBrightness(BRIGHTNESS);
  FastLED.setMaxPowerInVoltsAndMilliamps(5, 500);   // protects the power bank
  FastLED.clear(true);

  WiFi.mode(WIFI_STA);
  WiFi.setSleep(false);              // faster RSSI updates
  WiFi.begin(WIFI_SSID, WIFI_PASS);

  while (WiFi.status() != WL_CONNECTED) {
    showConnecting();
    delay(100);
  }
```

```
  Serial.print("Connected, IP: ");
  Serial.println(WiFi.localIP());
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) {
    showDisconnected();
    if (millis() - lastReconnect > 5000) {
      lastReconnect = millis();
      WiFi.disconnect();
      WiFi.begin(WIFI_SSID, WIFI_PASS);
    }
    delay(20);
    return;
  }

  int rssi = WiFi.RSSI();            // dBm, e.g. -45 (strong) .. -85 (weak)
  float target = (float)(rssi - RSSI_WEAK) / (float)(RSSI_STRONG - RSSI_WEAK);
  target = constrain(target, 0.0f, 1.0f);

  level += SMOOTHING * (target - level);   // exponential smoothing, no flicker

  // red (hue 0) -> green (hue 96) as the signal improves
  uint8_t hue = (uint8_t)(level * 96.0f);
  float lit = level * NUM_LEDS;
  int full = (int)lit;
  float frac = lit - full;

  for (int i = 0; i < NUM_LEDS; i++) {
    if (i < full) {
      leds[i] = CHSV(hue, 255, 255);
    } else if (i == full) {
      leds[i] = CHSV(hue, 255, (uint8_t)(frac * 255));   // smooth partial LED
    } else {
      leds[i] = CRGB(0, 0, 0);
    }
  }
  FastLED.show();

  static unsigned long lastPrint = 0;
  if (millis() - lastPrint > 500) {
    lastPrint = millis();
    Serial.printf("RSSI: %d dBm  level: %.2f\n", rssi, level);
  }
  delay(20);
}
```